

Министерство науки и высшего образования Российской Федерации
Южный федеральный университет
Физический факультет

КУРСОВОЙ ПРОЕКТ

по дисциплине «Администрирование ОС Linux»

**Тема: Использование NFS-сервера для сетевого
хранения данных**

Авторы (Группа):

Григорян А. А.

Карлашов А. В.

Кроливец И. С.

Проверил:

Тимошенко Павел Евгеньевич

Ростов-на-Дону, 2026

0 Содержание

Введение	3
1 Распределение ролей в группе и индивидуальный вклад	4
1.1 Инженер по автоматизации инфраструктуры — Григорян Ашот	4
1.2 Системный администратор / SRE — Карлашов Андрей . . .	5
1.3 Инженер по мониторингу и безопасности — Кроливец Иван	5
2 Теоретические основы технологии NFS	7
3 Проектирование архитектуры и сетевая изоляция	8
4 Автоматизация развёртывания и концепция IaC	10
4.1 Декларативное описание стека	10
4.2 Автоматизация жизненного цикла окружения	11
5 Настройка NFS-сервера и клиентов	12
6 Системная безопасность и hardening Linux-окружения	14
7 Тестирование, резервное копирование и CI/CD	15
Заключение	17

0 Введение

Современный этап развития корпоративной ИТ-инфраструктуры предъявляет всё более высокие требования к централизованному хранению и совместному использованию файловых ресурсов. В средах с большим числом узлов — будь то физические серверы, виртуальные машины или контейнеры — каждый узел не должен хранить собственную копию данных; необходима единая точка хранения, доступная всем участникам инфраструктуры одновременно.

Протокол NFS (Network File System), разработанный компанией Sun Microsystems в 1984 году и прошедший эволюцию вплоть до версии NFSv4.2, остаётся отраслевым стандартом сетевых файловых систем для Unix-подобных сред. Его нативная интеграция с ядром Linux, высокая производительность при работе с большим числом файлов и богатые возможности разграничения прав доступа делают NFS оптимальным выбором для задач горизонтального масштабирования хранилищ и организации совместной работы контейнеров с общими данными.

Однако ручная настройка NFS-сервера неизбежно порождает «дрейф конфигураций» (configuration drift) и ошибки воспроизводимости. Применение методологии Infrastructure as Code (IaC) совместно с инструментами контейнеризации Docker позволяет полностью автоматизировать жизненный цикл развёртывания, устранив человеческий фактор.

Цель данного проекта — комплексное проектирование, развёртывание, настройка и защита NFS-сервера в контейнеризированной среде под управлением ОС Linux, а также обеспечение безопасного сетевого доступа к нему для множества клиентских контейнеров.

1 Распределение ролей в группе и индивидуальный вклад

Разработка, тестирование и обеспечение безопасности сетевого файлового хранилища выполнялись проектной группой из трёх человек с чётким разграничением обязанностей и зон инженерной ответственности.

Участник	Роль	Основные задачи
Григорян Ашот	Инженер по автоматизации	Docker, Compose, bootstrap.sh, Dockerfile, Makefile
Карлашов Андрей	Системный администратор / SRE	Настройка NFS, монтирование, healthcheck, HA-тесты
Кроливец Иван	Инженер по безопасности	Hardening, UFW, Prometheus, Grafana, CI/CD-пайплайн

Таблица 1: Распределение ролей в проектной группе

1.1 Инженер по автоматизации инфраструктуры — Григорян Ашот

В зону ответственности инженера по автоматизации входила разработка декларативного описания всей инфраструктуры и автоматизация её жизненного цикла:

- Проектирование структуры Git-репозитория и настройка ветвления (main, feature/*, gh-pages);
- Разработка главного оркестрационного манифеста `docker-compose.yml` с описанием NFS-сервера, клиентских контейнеров и изолированных виртуальных сетей;
- Написание скрипта `bootstrap.sh` первичной подготовки ОС: установка зависимостей, создание директорий экспорта, настройка прав доступа;
- Сборка оптимизированных Dockerfile с применением принципа наименьших привилегий (non-root user) и многоэтапной сборки (multi-stage builds);

- Автоматизация управления стендом через Makefile: цели `up`, `down`, `test`, `clean`.

1.2 Системный администратор / SRE — Карлашов Андрей

Системный администратор сфокусировался на конфигурации сервисного уровня и обеспечении отказоустойчивости:

- Настройка NFS-сервера (`nfs-kernel-server` / NFS Ganesha): конфигурация файла `/etc/exports`, опций `rw`, `sync`, `root_squash`, `fsid`;
- Конфигурация нескольких клиентских контейнеров с автоматическим монтированием NFS-шары через Docker Volume (`driver: local`, `type: nfs`);
- Написание и интеграция скриптов `healthcheck` для проверки доступности NFS-экспорта;
- Проведение НА-тестов: имитация падения сервера, замер времени восстановления доступа к данным клиентами;
- Проверка блокировок файлов при одновременной записи из нескольких контейнеров.

1.3 Инженер по мониторингу и безопасности — Кроливец Иван

Инженер по безопасности обеспечивал многоуровневую защиту хранилища и наблюдаемость системы:

- Разработка скрипта `hardening.sh`: конфигурация UFW по принципу «запрещено всё, кроме явно разрешённого»; открыты только порты SSH (22), NFS (2049) для доверенных подсетей;
- Изоляция сети: NFS-трафик строго ограничен виртуальной сетью `storage-net`, внешний доступ к порту 2049 заблокирован;
- Развёртывание стека мониторинга Prometheus + Grafana с экспортером `node_exporter` и `cAdvisor` для сбора метрик контейнеров;

- Разработка дашбордов Grafana: нагрузка на NFS-шару, I/O дисковой подсистемы, состояние контейнеров;
- Настройка конвейера CI/CD в GitHub Actions: линтеры ShellCheck, yamllint, Hadolint; блокировка слияния при провале тестов.

2 Теоретические основы технологии NFS

NFS (Network File System) — клиент-серверный протокол, позволяющий монтировать удалённые файловые системы как локальные. В ядре Linux реализована поддержка NFSv3 и NFSv4/4.1/4.2. Архитектура NFS строится на модели RPC (Remote Procedure Call) с транспортом по протоколам TCP и UDP.

Версия NFSv4 принципиально отличается от предшественников: она использует единственный порт 2049/TCP, обеспечивает интегрированную безопасность через Kerberos или RPCSEC_GSS, поддерживает сессионную модель с управлением состоянием соединения и атомарные операции аренды (lease). NFSv4.1 добавляет параллельное расположение данных (pNFS), а NFSv4.2 — серверную копию файлов без участия клиента (server-side copy).

Ключевые параметры экспорта NFS, задаваемые в файле `/etc/exports`:

- `rw / ro` — разрешение записи или только чтения для клиента;
- `sync` — гарантия записи данных на диск до подтверждения клиенту (важно для целостности данных);
- `async` — более высокая производительность, но риск потери данных при сбое;
- `root_squash` — перемapping root-пользователя клиента в анонимный UID/GID (безопасность);
- `no_root_squash` — отключение mappingа (используется только в доверенных средах);
- `fsid=0` — обязателен для NFSv4, определяет корень псевдофайловой системы экспорта.

Сравнение NFS с альтернативными решениями. При выборе технологии хранения была проведена сравнительная оценка трёх подходов:

Технология	Преимущества	Недостатки	Вывод
NFS	Нативная поддержка в Linux, скорость, простая интеграция с Docker	Ограниченная отказоустойчивость, слабая защита без Kerberos	Оптимален
Samba (SMB)	Кросс-платформенность, интеграция с AD/LDAP	Выше накладные расходы в Linux, сложнее интеграция с Docker	Избыточен
GlusterFS	Распределённая архитектура, репликация, масштабирование	Высокая сложность настройки, требует от 3 узлов кластера	Избыточен

Таблица 2: Сравнение технологий сетевого файлового хранилища

3 Проектирование архитектуры и сетевая изоляция

Архитектура стенда построена по принципу минимальной связности компонентов. Все сервисы развёрнуты в Docker-контейнерах и взаимодействуют исключительно через изолированные виртуальные сети.

Состав кластера:

nfs-server Основной сервис хранения данных. Экспортирует директорию `/exports/shared` по протоколу NFSv4 на порту 2049. Примонтированный именованный том `nfs_data` обеспечивает персистентность данных.

nfs-client-1 Первый клиент. Автоматически монтирует NFS-шару через Docker Volume (`driver: local, type: nfs4`) при старте контейнера. Имитирует сервис приложения.

nfs-client-2 Второй клиент. Монтирует ту же NFS-шару, демонстрируя совместный доступ к данным из нескольких контейнеров одновременно.

monitoring Контейнер Prometheus + Grafana для сбора метрик нагрузки на NFS-сервер и дисковой подсистемы.

Сетевая topology. Безопасность сетевого контура реализована через виртуализацию Docker. Созданы две изолированные сети:

- **storage-net (bridge)** — приватная сеть для NFS-трафика. NFS-сервер и оба клиента подключены исключительно к этой сети. Внешний доступ к порту 2049 отсутствует.

- **monitoring-net** (bridge) — отдельная сеть для Prometheus/Grafana. Дашборды Grafana доступны на порту 3000 хост-машины с базовой HTTP-аутентификацией.

Сетевое разграничение гарантирует, что NFS-трафик (в том числе данные файлов) не может быть перехвачен другими контейнерами и не покидает виртуальный сегмент **storage-net**.

4 Автоматизация развёртывания и концепция IaC

Развёртывание инфраструктуры реализовано в рамках парадигмы IaC с помощью декларативного инструмента Docker Compose. Это полностью исключает дрейф конфигураций и гарантирует идентичность сред разработки, тестирования и промышленной эксплуатации.

4.1 Декларативное описание стека

Ниже приведён фрагмент конфигурационного файла `docker-compose.yml`, описывающий NFS-сервер и одного из клиентов:

```
1  services:
2    nfs-server:
3      image: erichough/nfs-server:latest
4      container_name: nfs-server
5      restart: always
6      privileged: true
7      environment:
8        NFS_EXPORT_0: '/exports/shared *(rw,sync,no_subtree_check,
9        fsid=0)'
10     volumes:
11       - nfs_data:/exports/shared
12     networks:
13       storage-net:
14         ipv4_address: 172.20.0.10
15
16     nfs-client-1:
17       image: alpine:3.19
18       container_name: nfs-client-1
19       restart: always
20       depends_on:
21         nfs-server:
22           condition: service_healthy
23       volumes:
24         - type: volume
25           source: nfs_shared
26           target: /mnt/shared
27       networks:
28         - storage-net
```

Листинг 1: Фрагмент docker-compose.yml

4.2 Автоматизация жизненного цикла окружения

Ниже приведена реализация скрипта первичной инициализации хост-системы:

```
1  #!/bin/bash
2  # Host initialization script
3  set -euo pipefail
4
5  EXPORT_DIR="${EXPORT_DIR:-/opt/nfs/shared}"
6  BACKUP_DIR="${BACKUP_DIR:-/opt/nfs/backups}"
7
8  # Export directories setup
9  for dir in "${EXPORT_DIR}" "${BACKUP_DIR}"; do
10 if [[ ! -d "${dir}" ]]; then
11 mkdir -p "${dir}"
12 chown nobody:nogroup "${dir}"
13 chmod 777 "${dir}"
14 echo "LOG: Created directory ${dir}"
15 fi
16 done
17
18 # Environment template verification
19 if [[ ! -f ".env" ]]; then
20 cp .env.example .env
21 echo "WARN: .env configuration initialized!"
22 fi
23
```

Листинг 2: Скрипт bootstrap.sh

5 Настройка NFS-сервера и клиентов

Конфигурация NFS-сервера определяется файлом `/etc/exports`, описывающим экспортируемые директории и права доступа для клиентов.

Конфигсистема файла `/etc/exports`:

```
1 /exports/shared 172.20.0.0/24(rw,sync,no_subtree_check,fsid
=0,root_squash)
2 /exports/shared 172.20.0.11(rw,sync,no_root_squash,fsid=0)
3
```

Листинг 3: Файл `/etc/exports`

Строка для подсети `172.20.0.0/24` включает `root_squash` — перемapping привилегированных запросов в анонимного пользователя, что снижает риск несанкционированного повышения привилегий. Для доверенного клиента `172.20.0.11` (тестовый контейнер) `no_root_squash` разрешает работу от `root` в изолированной среде.

Монтирование NFS через Docker Volume. Клиентские контейнеры монтируют NFS-шару напрямую через встроенный драйвер Docker Volume без установки дополнительных пакетов в образ:

```
1 volumes:
2 nfs_shared:
3 driver: local
4 driver_opts:
5 type: nfs4
6 o: "addr=172.20.0.10,rw,sync,nfsvers=4,hard,intr"
7 device: ":/exports/shared"
8
```

Листинг 4: Конфигурация Docker Volume для NFS

Контроль жизнеспособности (healthcheck). Для NFS-сервера настроен `healthcheck`, проверяющий доступность экспорта командой `showmount`:

```
1 healthcheck:
2 test: ["CMD", "showmount", "-e", "localhost"]
3 interval: 30s
4 timeout: 10s
5 retries: 5
6 start_period: 20s
7
```

Листинг 5: Healthcheck для NFS-сервера

6 Системная безопасность и hardening Linux-окружения

Многоуровневый периметр защиты охватывает сетевой стек ОС хоста, права доступа файловой системы и изоляцию контейнеров.

Межсетевой экран UFW. Скрипт `hardening.sh` переводит UFW в режим блокирования входящего трафика по умолчанию. Открыты только строго необходимые порты:

Порт / Протокол	Назначение	Ограничение
22/TCP	SSH — удалённое администрирование	Только admin-IP
2049/TCP	NFS — внутри storage-net	Подсеть 172.20.0.0/24
3000/TCP	Grafana — веб-интерфейс	localhost / admin-сеть
9090/TCP	Prometheus — метрики	Только localhost

Таблица 3: Правила межсетевого экрана UFW

Принцип наименьших привилегий в контейнерах:

- Все контейнеры, кроме `nfs-server`, запускаются от непривилегированного пользователя (`USER appuser`);
- `nfs-server` требует `privileged: true` для работы с ядерными модулями NFS — это задокументированное исключение;
- `cap_drop: [ALL]` применяется ко всем клиентским контейнерам;
- Корневая ФС клиентов монтируется в режиме `read_only: true`.

7 Тестирование, резервное копирование и CI/CD

Стратегия интеграционного тестирования. Валидация развёрнутого стенда осуществляется скриптом `test_nfs.sh`, который проверяет четыре уровня работоспособности:

1. **Сетевая доступность** — опрашивает экспортируемые директории NFS-сервера командой `showmount -e nfs-server` с таймаутом 60 секунд.
2. **Монтирование контейнеров** — проверяет, что NFS-шара примонтирована в каждом клиентском контейнере командой `df -h | grep nfs`.
3. **Совместный доступ** — записывает файл из `nfs-client-1` и читает его из `nfs-client-2`, проверяя идентичность содержимого.
4. **НА-тест** — принудительно останавливает `nfs-server`, ожидает перезапуска и проверяет восстановление монтирования.

Резервное копирование. Скрипт `backup.sh` реализует «горячее» резервное копирование содержимого NFS-экспорта без остановки сервиса. Применяется утилита `rsync` с флагами `-archive -checksum`, обеспечивающими инкрементальное копирование только изменённых файлов. Ротация архивов удаляет копии старше 7 дней:

```
1 find "${BACKUP_DIR}" -type f -name "*.tar.gz" -mtime +7 -
2 exec rm {} \;
```

Листинг 6: Ротация архивов в `backup.sh`

Конвейер CI/CD (GitHub Actions). Проект интегрирован с облачным конвейером GitHub Actions. Пайплайн `pr-validation.yml` запускается автоматически при открытии Pull Request и включает следующие этапы:

- **yamllint** — проверка синтаксиса всех YAML-файлов проекта;

- **ShellCheck** — аудит bash-скриптов на логические ошибки и соответствие POSIX;
- **Hadolint** — проверка Dockerfile на оптимизацию слоёв сборки;
- **Динамические тесты** — развёртывание стенда в эфемерном Ubuntu-раннере и запуск `test_nfs.sh`.

Слияние кода в ветку `main` блокируется до успешного прохождения всех этапов конвейера.

7 Заключение

В ходе выполнения данного курсового проекта был успешно спроектирован, развёрнут и защищён отказоустойчивый сервис сетевого файлового хранилища на базе протокола NFS в контейнеризированной среде под управлением ОС Linux.

Применение методологии Infrastructure as Code и инструментов Docker Compose позволило полностью автоматизировать все этапы: подготовку окружения, настройку экспортов, монтирование в клиентских контейнерах и проведение интеграционных тестов.

Реализованная многоуровневая система безопасности — изоляция сети `storage-net`, правила UFW по принципу «запрещено всё», принцип наименьших привилегий в контейнерах — обеспечила высокий уровень защищённости хранимых данных от несанкционированного доступа.

Развёрнутый стек мониторинга Prometheus + Grafana с кастомными дашбордами предоставляет полную наблюдаемость (observability) работы NFS-сервера в реальном времени. Интеграция с CI/CD пайплайном GitHub Actions гарантирует стабильность инфраструктуры на протяжении всего жизненного цикла её эксплуатации.

Результат	Статус
Развёртывание NFS Replica Set в Docker Compose	✓ Выполнено
Автоматизация bootstrap.sh и Makefile	✓ Выполнено
Многоуровневый hardening (UFW, cap_drop, read_only)	✓ Выполнено
Интеграционные тесты и НА-тестирование	✓ Выполнено
Мониторинг Prometheus + Grafana	✓ Выполнено
CI/CD пайплайн GitHub Actions	✓ Выполнено

Таблица 4: Итоговые результаты проекта